



75 SQL Questions to Crack Any Job Interview — With Answers & Queries

 **75 real interview questions, each with the exact query that answers it.** Organized by category so you can drill your weak spots directly instead of grinding randomly.

The Set

Section	Questions	Focus
 Basics & Filtering	#1 – 12	Core querying under time pressure
 Joins	#13 – 24	Combining tables correctly
 Aggregation	#25 – 36	Turning rows into business answers
 Window Functions	#37 – 51	Per-row context — the section that gets you hired
 Subqueries & CTEs	#52 – 61	Multi-step, readable logic
 Advanced / Business Scenarios	#62 – 75	Real-world messy questions

Basics & Filtering — #1 to #12

1. Find all employees earning more than \$70,000.

```
SELECT * FROM employees WHERE salary > 70000;
```

2. Find the 5 most recent orders.

```
SELECT * FROM orders ORDER BY order_date DESC LIMIT 5;
```

3. Find all customers whose email is a Gmail address.

```
SELECT * FROM customers WHERE email LIKE '%gmail.com';
```

4. Find all orders with status either 'shipped' or 'delivered'.

```
SELECT * FROM orders WHERE status IN ('shipped', 'delivered');
```

5. Find customers with a missing phone number.

```
SELECT * FROM customers WHERE phone IS NULL;
```

6. Find all distinct product categories.

```
SELECT DISTINCT category FROM products;
```

7. Find orders placed in the last 30 days.

```
SELECT * FROM orders WHERE order_date >= CURRENT_DATE - INTERVAL '30 days';
```

8. Find products priced between \$50 and \$200.

```
SELECT * FROM products WHERE price BETWEEN 50 AND 200;
```

9. Sort employees by department, then by salary descending within each department.

```
SELECT * FROM employees ORDER BY department ASC, salary DESC;
```

10. Rename the output column 'name' to 'Full Name'.

```
SELECT name AS "Full Name" FROM employees;
```

11. Find the 3 highest-priced products.

```
SELECT * FROM products ORDER BY price DESC LIMIT 3;
```

12. Find employees whose name starts with "A".

```
SELECT * FROM employees WHERE name LIKE 'A%';
```

Joins — #13 to #24

13. List each order alongside the customer's name.

```
SELECT o.order_id, c.name FROM orders o INNER JOIN customer  
s c ON o.customer_id=c.customer_id;
```

14. List every customer, and their order id if they have one.

```
SELECT c.name, o.order_id FROM customers c LEFT JOIN orders  
o ON c.customer_id=o.customer_id;
```

15. Find customers who have never placed an order.

```
SELECT c.name FROM customers c LEFT JOIN orders o ON c.cust  
omer_id=o.customer_id WHERE o.order_id IS NULL;
```

16. List every employee with their manager's name.

```
SELECT e.name AS employee, m.name AS manager FROM employees  
e LEFT JOIN employees m ON e.manager_id=m.employee_id;
```

17. Reconcile two tables, showing rows that exist in only one of them.

```
SELECT * FROM table_a a FULL OUTER JOIN table_b b ON a.id=  
b.id WHERE a.id IS NULL OR b.id IS NULL;
```

18. Find pairs of students in the same class.

```
SELECT a.name, b.name FROM students a JOIN students b ON a.
```

```
class_id=b.class_id AND a.id < b.id;
```

19. List products that have never been ordered.

```
SELECT p.name FROM products p LEFT JOIN order_items oi ON  
p.product_id=oi.product_id WHERE oi.order_id IS NULL;
```

20. Count how many orders each customer has placed, including customers with zero orders.

```
SELECT c.name, COUNT(o.order_id) AS order_count FROM custom  
ers c LEFT JOIN orders o ON c.customer_id=o.customer_id GRO  
UP BY c.name;
```

21. Join three tables: orders, customers, and products (via order_items).

```
SELECT o.order_id, c.name, p.product_name  
FROM orders o  
JOIN customers c ON o.customer_id=c.customer_id  
JOIN order_items oi ON o.order_id=oi.order_id  
JOIN products p ON oi.product_id=p.product_id;
```

22. Find employees who work in the same department as "Priya".

```
SELECT e2.name FROM employees e1 JOIN employees e2 ON e1.de  
partment=e2.department  
WHERE e1.name='Priya' AND e2.name != 'Priya';
```

23. Find all orders along with the total item count per order.

```
SELECT o.order_id, COUNT(oi.item_id) AS item_count FROM ord  
ers o JOIN order_items oi ON o.order_id=oi.order_id GROUP B  
Y o.order_id;
```

24. List customers and their most recent order date (NULL if none).

```
SELECT c.name, MAX(o.order_date) AS last_order FROM custome  
rs c LEFT JOIN orders o ON c.customer_id=o.customer_id GROU
```

```
P BY c.name;
```

● Aggregation — #25 to #36

25. Total revenue per region.

```
SELECT region, SUM(revenue) FROM sales GROUP BY region;
```

26. Regions where average revenue exceeds \$5,000.

```
SELECT region, AVG(revenue) FROM sales GROUP BY region HAVING AVG(revenue) > 5000;
```

27. Count of distinct customers per product category.

```
SELECT category, COUNT(DISTINCT customer_id) FROM orders GROUP BY category;
```

28. Count of completed vs cancelled orders per region, side by side.

```
SELECT region,
    SUM(CASE WHEN status='completed' THEN 1 ELSE 0 END) AS completed,
    SUM(CASE WHEN status='cancelled' THEN 1 ELSE 0 END) AS cancelled
FROM orders GROUP BY region;
```

29. The second-highest salary, handling ties safely.

```
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);
```

30. Min, max, and average salary per department.

```
SELECT department, MIN(salary), MAX(salary), AVG(salary) FROM employees GROUP BY department;
```

31. Top 10 products by total revenue.

```
SELECT product_id, SUM(quantity*price) AS revenue FROM order_items GROUP BY product_id ORDER BY revenue DESC LIMIT 10;
```

32. Number of orders placed per month.

```
SELECT DATE_TRUNC('month', order_date) AS month, COUNT(*) FROM orders GROUP BY 1 ORDER BY 1;
```

33. Find duplicate email addresses and how many times each occurs.

```
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
```

34. Total sales and order count in a single query.

```
SELECT SUM(amount) AS total_sales, COUNT(*) AS order_count FROM orders;
```

35. Average order value per customer, only for customers with 5+ orders.

```
SELECT customer_id, AVG(amount) FROM orders GROUP BY customer_id HAVING COUNT(*) >= 5;
```

36. Percentage each region contributes to total revenue.

```
SELECT region, SUM(revenue)*100.0/(SELECT SUM(revenue) FROM sales) AS pct_of_total FROM sales GROUP BY region;
```

Window Functions — #37 to #51

37. Rank employees by salary, highest first.

```
SELECT name, salary, RANK() OVER (ORDER BY salary DESC) AS rn FROM employees;
```

38. Rank employees by salary WITHIN each department.

```
SELECT name, department, salary, RANK() OVER (PARTITION BY
department ORDER BY salary DESC) AS dept_rank FROM employee
s;
```

39. Difference between RANK and DENSE_RANK, shown side by side.

```
SELECT name, salary, RANK() OVER (ORDER BY salary DESC) AS
rnk, DENSE_RANK() OVER (ORDER BY salary DESC) AS drnk FROM
employees;
```

40. Running total of daily sales.

```
SELECT order_date, amount, SUM(amount) OVER (ORDER BY order
_date) AS running_total FROM orders;
```

41. 7-day rolling average of daily visits.

```
SELECT visit_date, visits, AVG(visits) OVER (ORDER BY visit
_date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS rolling_
avg FROM daily_traffic;
```

42. Month-over-month revenue change.

```
SELECT month, revenue, revenue - LAG(revenue) OVER (ORDER B
Y month) AS change FROM monthly_sales;
```

43. Days until each customer's next order.

```
SELECT customer_id, order_date, LEAD(order_date) OVER (PART
ITION BY customer_id ORDER BY order_date) - order_date AS d
ays_to_next FROM orders;
```

44. Assign each student to a quartile based on score.

```
SELECT name, score, NTILE(4) OVER (ORDER BY score DESC) AS
quartile FROM students;
```

45. Show each department's top salary next to every employee row.

```
SELECT name, department, salary, FIRST_VALUE(salary) OVER
(PARTITION BY department ORDER BY salary DESC) AS top_salary_in_dept FROM employees;
```

46. What percentile does each salary fall into?

```
SELECT name, salary, PERCENT_RANK() OVER (ORDER BY salary)
AS pct_rank FROM employees;
```

47. Return only each customer's most recent order.

```
WITH ranked AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC) AS rn FROM orders
)
SELECT * FROM ranked WHERE rn = 1;
```

48. Top 3 highest-paid employees per department.

```
WITH ranked AS (
    SELECT *, RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk FROM employees
)
SELECT * FROM ranked WHERE rnk <= 3;
```

49. Each product's price compared to its category average.

```
SELECT product_id, category, price, price - AVG(price) OVER
(PARTITION BY category) AS diff_from_avg FROM products;
```

50. Each row's percent of the running total so far.

```
SELECT order_date, amount,
    SUM(amount) OVER (ORDER BY order_date) * 100.0 / SUM(amount) OVER () AS pct_of_total_so_far
FROM orders;
```

51. Find users active on 3 or more consecutive days.


```
WITH numbered AS (
    SELECT user_id, login_date,
           login_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date))::int AS grp
    FROM logins
)
SELECT user_id, COUNT(*) FROM numbered GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

● Subqueries & CTEs — #52 to #61

52. Employees earning above their own department's average.

```
SELECT name, salary FROM employees e1
WHERE salary > (SELECT AVG(salary) FROM employees e2 WHERE
e2.department=e1.department);
```

53. Customers who have placed at least one order over \$1,000.

```
SELECT name FROM customers c WHERE EXISTS (SELECT 1 FROM orders o
WHERE o.customer_id=c.customer_id AND o.amount>1000);
```

54. Month-over-month revenue growth using a CTE.

```
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date) AS mo, SUM(amount)
    AS revenue FROM orders GROUP BY 1
)
SELECT mo, revenue, LAG(revenue) OVER (ORDER BY mo) AS prev
_month FROM monthly;
```

55. Full management hierarchy from the CEO down.

```
WITH RECURSIVE org AS (
    SELECT employee_id, name, manager_id, 1 AS level FROM employees
    WHERE manager_id IS NULL
```

```

UNION ALL
SELECT e.employee_id, e.name, e.manager_id, o.level+1 FROM employees e JOIN org o ON e.manager_id=o.employee_id
)
SELECT * FROM org ORDER BY level;

```

56. Customers whose total spend is above the overall average customer spend.

```

WITH totals AS (SELECT customer_id, SUM(amount) AS total FROM orders GROUP BY customer_id)
SELECT * FROM totals WHERE total > (SELECT AVG(total) FROM totals);

```

57. Products never ordered, using NOT EXISTS.

```

SELECT name FROM products p WHERE NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.product_id=p.product_id);

```

58. Top 3 departments by average salary, using a CTE.

```

WITH dept_avg AS (SELECT department, AVG(salary) AS avg_sal FROM employees GROUP BY department)
SELECT * FROM dept_avg ORDER BY avg_sal DESC LIMIT 3;

```

59. Filter using a subquery in the FROM clause.

```

SELECT * FROM (SELECT *, RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rnk FROM employees) t WHERE rnk <= 3;

```

60. Customers who ordered in January but not February.

```

SELECT DISTINCT customer_id FROM orders WHERE order_date BETWEEN '2026-01-01' AND '2026-01-31'
AND customer_id NOT IN (SELECT customer_id FROM orders WHERE order_date BETWEEN '2026-02-01' AND '2026-02-28');

```

61. Chain two CTEs: totals, then rank those totals.

```
WITH totals AS (SELECT customer_id, SUM(amount) AS total FROM orders GROUP BY customer_id),
ranked AS (SELECT *, RANK() OVER (ORDER BY total DESC) AS rnk FROM totals)
SELECT * FROM ranked WHERE rnk <= 5;
```

● Advanced / Business Scenarios — #62 to #75

62. Bucket customers into age groups.

```
SELECT customer_id,
CASE WHEN age < 25 THEN '18-24' WHEN age < 35 THEN '25-34' WHEN age < 50 THEN '35-49' ELSE '50+' END AS age_bucket
FROM customers;
```

63. Random sample of 1,000 customers.

```
SELECT * FROM customers ORDER BY RANDOM() LIMIT 1000;
```

64. Cohort retention: users active in month 1 vs month 2 after signup.

```
WITH first_month AS (SELECT user_id, DATE_TRUNC('month', MIN(activity_date)) AS cohort_month FROM activity GROUP BY user_id)
SELECT f.cohort_month, COUNT(DISTINCT a.user_id) AS active_users
FROM first_month f JOIN activity a ON f.user_id=a.user_id
WHERE DATE_TRUNC('month', a.activity_date) = f.cohort_month
+ INTERVAL '1 month'
GROUP BY f.cohort_month;
```

65. Z-score of each customer's revenue.

```
SELECT customer_id, revenue, (revenue - AVG(revenue) OVER ()) / NULLIF(STDDEV(revenue) OVER (), 0) AS revenue_zscore FROM
```

```
FROM customer_revenue;
```

66. One-hot-style encoding of purchase categories per customer.

```
SELECT customer_id,  
       MAX(CASE WHEN category='Electronics' THEN 1 ELSE 0 END) AS  
       bought_electronics,  
       MAX(CASE WHEN category='Clothing' THEN 1 ELSE 0 END) AS  
       bought_clothing  
FROM orders GROUP BY customer_id;
```

67. Products frequently bought together (market basket).

```
SELECT a.product_id AS product_a, b.product_id AS product_  
b, COUNT(*) AS times_together  
FROM order_items a JOIN order_items b ON a.order_id=b.order_  
_id AND a.product_id < b.product_id  
GROUP BY a.product_id, b.product_id ORDER BY times_together  
DESC;
```

68. Missing-data audit across multiple columns in one query.

```
SELECT  
       SUM(CASE WHEN age IS NULL THEN 1 ELSE 0 END) AS missing_a  
       ge,  
       SUM(CASE WHEN income IS NULL THEN 1 ELSE 0 END) AS missin  
       g_income,  
       COUNT(*) AS total_rows  
FROM customers;
```

69. Flag outliers using the IQR method.

```
WITH stats AS (  
    SELECT *,  
           PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY revenue) O  
           VER() AS q1,  
           PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY revenue) O  
           VER() AS q3
```

```

FROM customer_revenue
)
SELECT *, (revenue < q1-1.5*(q3-q1) OR revenue > q3+1.5*(q3-q1)) AS is_outlier FROM stats;

```

70. Classify customers as New, Returning, or Churned.

```

WITH stats AS (SELECT customer_id, MIN(order_date) AS first_order, MAX(order_date) AS last_order FROM orders GROUP BY customer_id)
SELECT customer_id,
CASE
WHEN first_order >= CURRENT_DATE - INTERVAL '30 days' THEN 'New'
WHEN last_order < CURRENT_DATE - INTERVAL '90 days' THEN 'Churned'
ELSE 'Returning'
END AS status
FROM stats;

```

71. Monthly DAU/MAU stickiness ratio.

```

WITH dau AS (SELECT DATE_TRUNC('month', activity_date) AS mo, activity_date, COUNT(DISTINCT user_id) AS d FROM activity GROUP BY 1,2),
mau AS (SELECT DATE_TRUNC('month', activity_date) AS mo, COUNT(DISTINCT user_id) AS m FROM activity GROUP BY 1)
SELECT d.mo, AVG(d.d)*1.0/MAX(mau.m) AS stickiness FROM dau d JOIN mau ON d.mo=mau.mo GROUP BY d.mo;

```

72. Find every date with zero activity within the dataset's range.

```

WITH all_dates AS (SELECT generate_series(MIN(activity_date), MAX(activity_date), '1 day') AS dt FROM attendance)
SELECT dt FROM all_dates WHERE dt NOT IN (SELECT DISTINCT activity_date FROM attendance);

```

73. Turn monthly rows into one row per product with a column per month (pivot).

```
SELECT product_id,  
       SUM(CASE WHEN month='Jan' THEN amount ELSE 0 END) AS jan,  
       SUM(CASE WHEN month='Feb' THEN amount ELSE 0 END) AS feb  
FROM sales GROUP BY product_id;
```

74. A/B test: click-through rate per variant.

```
SELECT variant, SUM(clicked)*1.0/COUNT(*) AS ctr FROM exper  
iment_events GROUP BY variant;
```

75. Friend-of-friend suggestions ranked by mutual connections.

```
SELECT f2.friend_id AS suggested, COUNT(*) AS mutual_friend  
s  
FROM friends f1 JOIN friends f2 ON f1.friend_id=f2.user_id  
WHERE f1.user_id=123 AND f2.friend_id != 123  
AND f2.friend_id NOT IN (SELECT friend_id FROM friends WHER  
E user_id=123)  
GROUP BY f2.friend_id ORDER BY mutual_friends DESC;
```

Self-Score

Range	What it means
60–75 correct	Interview-ready across every level
40–59 correct	Strong on fundamentals — drill window functions and advanced scenarios
20–39 correct	Core querying is solid, joins/aggregation need another pass
Under 20	Start with the SQL Prep Sheet (Zero to Hired) before this set

Before You Walk In...

Section	Can you do this cold?
Basics & Joins	Write an anti-join and a 3-table join without hesitating

Section	Can you do this cold?
Aggregation	Build a conditional aggregation pivot (SUM+CASE) from scratch
Window Functions	Explain RANK vs DENSE_RANK, write a rolling average and a top-N-per-group query
Subqueries/CTEs	Write a recursive CTE and chain two CTEs together
Advanced	Compute a z-score, flag IQR outliers, and classify customer status — all in SQL